SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

Email: umhackathon@um.edu.my

1. Introduction

Kira2 Je is a strategic solution for small Malaysian F&B businesses that have already generated digitised revenue data through their Point-of-Sale (POS) systems, but their expense data remains analogue, scattered across paper invoices, supplier delivery orders and unsorted receipts. This gap in the system means owners cannot see the true cost-versus-revenue picture of their business. They can see what they sold, but not what each sale actually cost them.

Kira2 Je addresses this gap by accepting POS sales data as an XLSX or CSV file upload and ingests expense invoices through three frictionless channels, which are, the in-app camera, WhatsApp bot and drag-and-drop file upload. The structured outputs of both pipelines are sent to OpenAI's GPT-4o-mini, which acts as the system's reasoning engine, producing item-level margin analysis, trend detection, cannibalisation alerts and ranked recommendations with estimated Ringgit Malaysia (RM) impact.

The product name is colloquial Bahasa Malaysia for "just roughly calculate," a knowing reference to how most small F&B owners currently handle finances. Kira2 Je replaces that guesswork with structured analysis, without forcing the owner to learn accounting.

1.1 Purpose

This System Analysis Document highlights the technical scope and design-related decisions behind the development of Kira2 Je for UM Hackathon 2026. The key elements of the system covered in this document are as follows:

1. **Architecture**:
   a. Kira2 Je is a web application with a separable service-layer design. The frontend, backend, OCR pipeline and the OpenAI's GPT-4o-mini reasoning service operate as independent components communicating over well-defined APIs.

2. **Data Flows:**
   a. Two parallel data ingestion channels (POS XLSX/CSV upload and multi-mode invoice capture) feed a single normalised data store. Structured records are then composed into prompts and dispatched to the model, whose JSON responses are parsed and rendered as dashboard insights, reports and what-if simulations.

3. **Model Process:**
   a. The end-to-end workflow covers the core features: uploading POS data, scanning or submitting invoices, generating an analysis report, surfacing cannibalisation alerts, and answering what-if questions through a structured simulator.

4. **Role of Reference:**
   a. This document is a reference for our development team, the QA team and the UM Hackathon 2026 judges, articulating the high-level architecture, design decisions and the central role of OpenAI's GPT-4o-mini in the product.

1.2 Background

Malaysia has a large and structurally significant base of small food and beverage (F&B) businesses, including independent restaurants, cafés, kopitiams, small chains, and catering operators. According to the Department of Statistics Malaysia, there were 136,453 F&B service establishments operating in 2022, with food services alone accounting for 107,129 establishments and generating RM99.0 billion in gross output (Department of Statistics Malaysia [DOSM], 2024). A meaningful portion of these businesses have adopted cloud-based point-of-sale (POS) systems, as reflected by platforms such as StoreHub, which serves over 18,000 businesses across Southeast Asia, and Slurp!, which is used by more than 2,600 outlets in Malaysia and Singapore (StoreHub, 2026; Slurp Retail Tech Sdn. Bhd., 2026). While this adoption enables efficient capture and export of transactional sales data, it does not extend to cost attribution at the item level. Cost data remains largely unstructured, typically recorded in paper-based or PDF invoices, limiting its integration into analytical workflows.

This disconnect is further reinforced by operational practices within the sector. Supplier transactions frequently rely on physical documentation and manual verification, a gap highlighted by the Malaysia Digital Economy Corporation in its discussion of the National E-Invoicing initiative, which aims to reduce paper-based processes and manual data entry (Malaysia Digital Economy Corporation [MDEC], 2026). As a result, despite partial digitalisation in sales systems, downstream financial analysis remains constrained. SME Corporation Malaysia reports that approximately 77% of Malaysian SMEs operate at a basic level of digitalisation, with only 6.3% leveraging data analytics capabilities (SME Corporation Malaysia, 2026). Consequently, business owners lack the ability to systematically map costs to revenue at a granular level, often relying on heuristic or approximate calculations rather than structured analysis. This phenomenon reflects a broader pattern of underutilised data infrastructure within the SME segment.

The target market for Kira2 Je is therefore defined by this structural gap. It excludes informal micro-operators such as street vendors, who typically fall outside formal financial record-keeping systems, as well as large enterprises that already possess mature analytics and business intelligence capabilities. Large firms account for 62.58% of revenue share in Malaysia's digital transformation market, indicating substantial investment in internal data systems and advanced tooling (Mordor Intelligence, 2026). Instead, Kira2 Je is positioned for the "digitised middle" segment: businesses that have already adopted digital sales infrastructure but lack the analytical layer required to transform raw data into actionable financial insights.

1. **Previous Version:**
    a. Kira2 Je is a new product with no prior version. The current alternative for our target users is a patchwork of manual methods: paper ledger books, ad-hoc Excel sheets, photographs of invoices stored in a phone gallery, or generic accounting software that does not perform cost-revenue matching at the menu-item level. None of these alternatives reasons about the data, they only record it. Kira2 Je's contribution is moving from recording to reasoning.

2. **Changes in Major Architectural Components and New Capabilities in this Version:**
    a. Cost-revenue matching at the line-item level. The model maps expense line items (e.g., "20kg chicken breast from Supplier X") to menu items ("nasi ayam," "chicken chop," "chicken rice") rather than treating them as undifferentiated "food cost."
    b. True per-item profitability after ingredient cost, platform commission (Grab, ShopeeFood) and overhead allocation.
    c. Multi-channel invoice capture (in-app camera, WhatsApp bot, drag-and-drop file upload) feeding a single OCR pipeline, eliminating the manual data-entry barrier that breaks every other tool in this segment.
    d. Cannibalisation detection, which can be unlocked after roughly three months of accumulated data. This helps identify when a new menu item is pulling sales from existing items rather than generating net new revenue.

e. A what-if simulator that lets owners ask hypothetical questions ("What if I raise teh tarik by RM0.50?") and receive projected impact analysis in RM.

1.3 Target Stakeholder

Stakeholders	Roles	Expectations
F&B Business Owner	Primary user. Uploads POS sales data, scans or forwards invoices, reviews the analysis report and acts on recommendations.	Low-friction data entry, plain-language insights in BM or English, recommendations with RM impact estimates, no accounting jargon.
Front-of-House / Operations Staff	Secondary user. Assists with day-to-day invoice scanning, particularly when invoices arrive with goods deliveries.	Camera scan and WhatsApp flow that take seconds, no login confusion, clear confirmation that an invoice was captured.
Suppliers (indirect)	Generate the physical and digital invoices that flow into the OCR pipeline.	No direct interaction. The system must tolerate varied invoice formats: BM, English, mixed languages, handwriting, printed and PDF.

2. System Architecture & Design

2.1 High Level Architecture

2.1.1. Overview

Type	Details
System	Web application (mobile-responsive). Designed for desktop browsers and mobile browsers; the camera and drag-and-drop flows work from a phone browser without requiring a native install.
Architecture	Cloud-deployed client-server with an AI API service layer. Independent functional units: Frontend client, Backend API, OCR Service, GPT-4o-mini Service, WhatsApp Webhook, Data Storager.
Channel Surface	1) Web app (React, mobile-responsive). 2) WhatsApp bot for invoice photo submission. 3) File upload zone for e-invoices (PDF, image).

Kira2 Je is structured as a client-server web application with a dedicated reasoning service layer. The web client communicates with a Backend API that orchestrates four downstream services: an OCR Service for invoice extraction, a GPT-4o-mini service that wraps OpenAI's API and constructs all reasoning prompts, a WhatsApp Webhook Service that receives photos forwarded from a WhatsApp bot number, and a data storage layer (PostgreSQL) that stores users, businesses, sales data, expenses, parsed invoices and historical reports. All services share the same database, allowing the GPT-4o-mini to perform multi-month trend and cannibalisation analysis.

2.1.2 LLM as Service Layer

OpenAI's GPT-4o-mini API serves as the core reasoning engine of the architecture. It powers cost–revenue matching, margin analysis, trend detection, cannibalisation detection, and what-if simulations, making it central to how the product functions. Without it, Kira2 Je does not operate as an intelligent system and instead reduces to a spreadsheet.

Concretely, the model is responsible for:

- Loading the appropriate prompt template based on the user's intent (analysis, OCR extraction, what-if)
- Injecting the user's language preference (BM or English), the structured POS JSON, the structured invoice JSON and any available historical context
- Enforcing token-budget limits before dispatch
- Calling OpenAI's API and parsing the JSON response
- Returning typed data to the Backend API for data storage and rendering.

2.1.3 Dependency Diagram

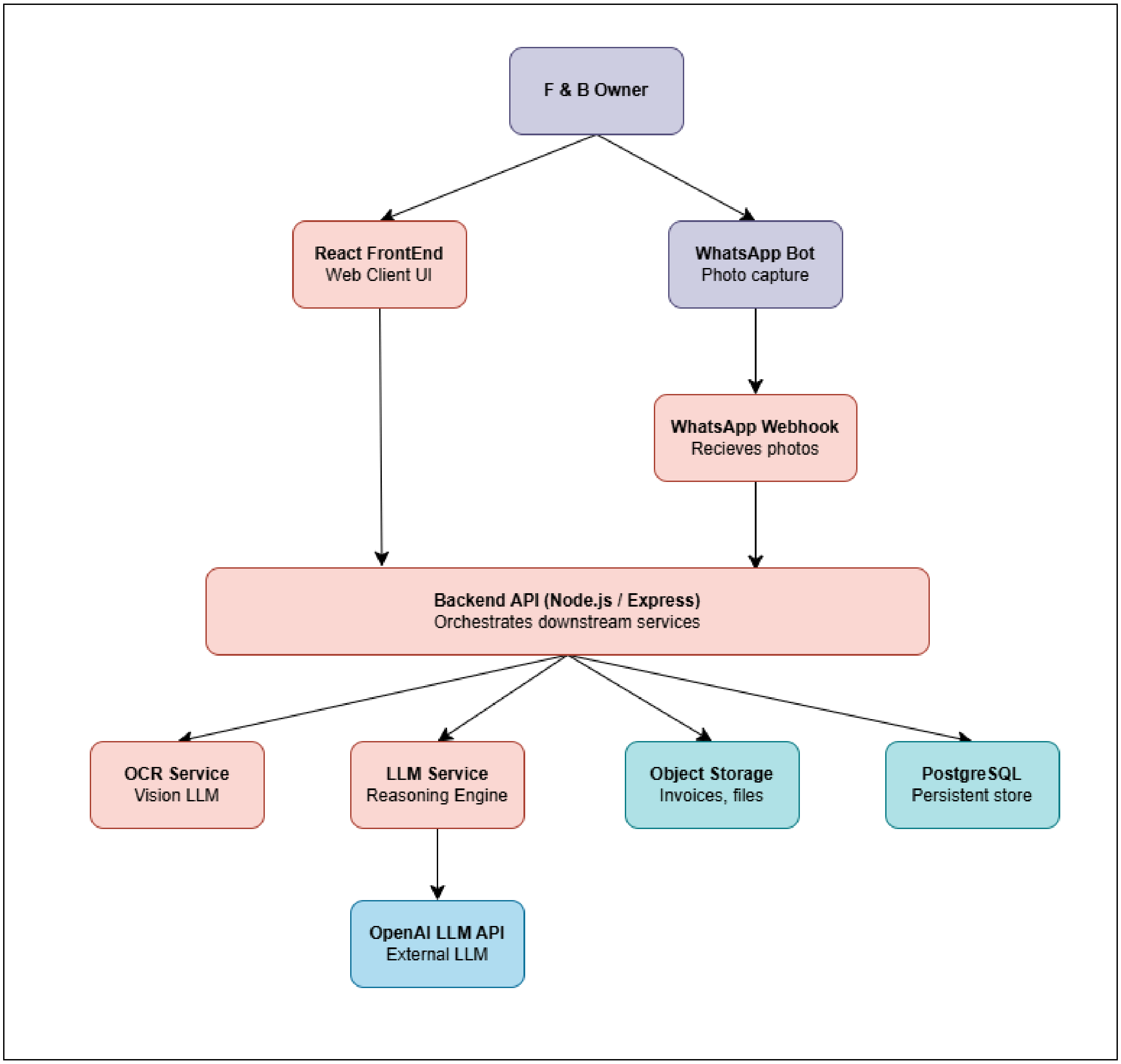

Figure 2.1.3a: Dependency Diagram

2.1.4 Prompt Construction & Context Window

The GPT-4o-mini service constructs every prompt from a versioned template. For each call, the following is injected into the context window:

- User's language preference (BM or English) was applied to all generated text fields in the response.
- Structured POS sales data as compact JSON (item, quantity, price, timestamp, payment method, platform). Truncation rules apply when the dataset exceeds the model's effective context budget - see Token Limits below.
- Structured expense data as compact JSON (supplier, line items, quantities, unit costs, totals, dates).
- Historical context allows a rolling summary of the previous one to three months of analysis reports. This is included only for trend and cannibalisation prompts. For first-time users this field is null.
- An explicit instruction to return JSON conforming to a defined schema, so the response can be parsed and rendered without natural-language post-processing.

2.1.5 Response Parsing & Hand-off

The GPT-4o-mini service expects a strict JSON response. The response is validated against the prompt's schema; on validation failure the service triggers a single retry with an explicit "return JSON only" reminder, then degrades gracefully by returning a structured error to the Backend API. Successful responses are persisted to the Reports table and returned to the client for rendering.

2.1.6 Token Limits, Truncation & Chunking

- All inputs are converted to compact JSON before injection - no raw xlsx or raw OCR text reaches the GPT-4o-mini model.
- POS data is aggregated to item-day granularity for the analysis prompt; raw transactional rows are kept in PostgreSQL but not sent to the model.
- If the aggregated payload exceeds the configured token budget, sales are chunked by category (food, beverage, set, add-on) and analysed in successive calls whose JSON outputs are merged.
- Invoice OCR results are bound to the line-item structure - supplier-side marketing text and headers are stripped before being forwarded.
- Repeated identical queries within a 24-hour window are served from a response cache to avoid redundant token spend.

2.1.7 Major API Calls Between Components

- **Frontend → Backend API:** POST /api/pos/upload (multipart xlsx); POST /api/invoices/upload (image or PDF); GET /api/reports/current; POST /api/whatif (question text).
- **Backend → OCR Service:** POST /ocr/extract with the invoice image and a structured-extraction prompt; returns parsed JSON with a confidence score.
- **Backend → GPT-4o-mini Service:** POST /glm/analyse, /glm/whatif. The GPT-4o-mini model is the only component that holds OpenAI API credentials.
- **GPT-4o-mini Service → OpenAI's GPT-4o-mini API:** POST to the OpenAI completion endpoint with the constructed prompt and JSON response format.
- **WhatsApp Webhook → Backend API:** POST /api/invoices/whatsapp with the user's WhatsApp identifier and the photo URL; the backend then forwards the photo to the OCR Service.
- **Backend → PostgreSQL:** Persistent reads/writes for users, businesses, POS uploads, invoice line items, ingredient mappings and report history.

2.1.8 Sequence Diagram

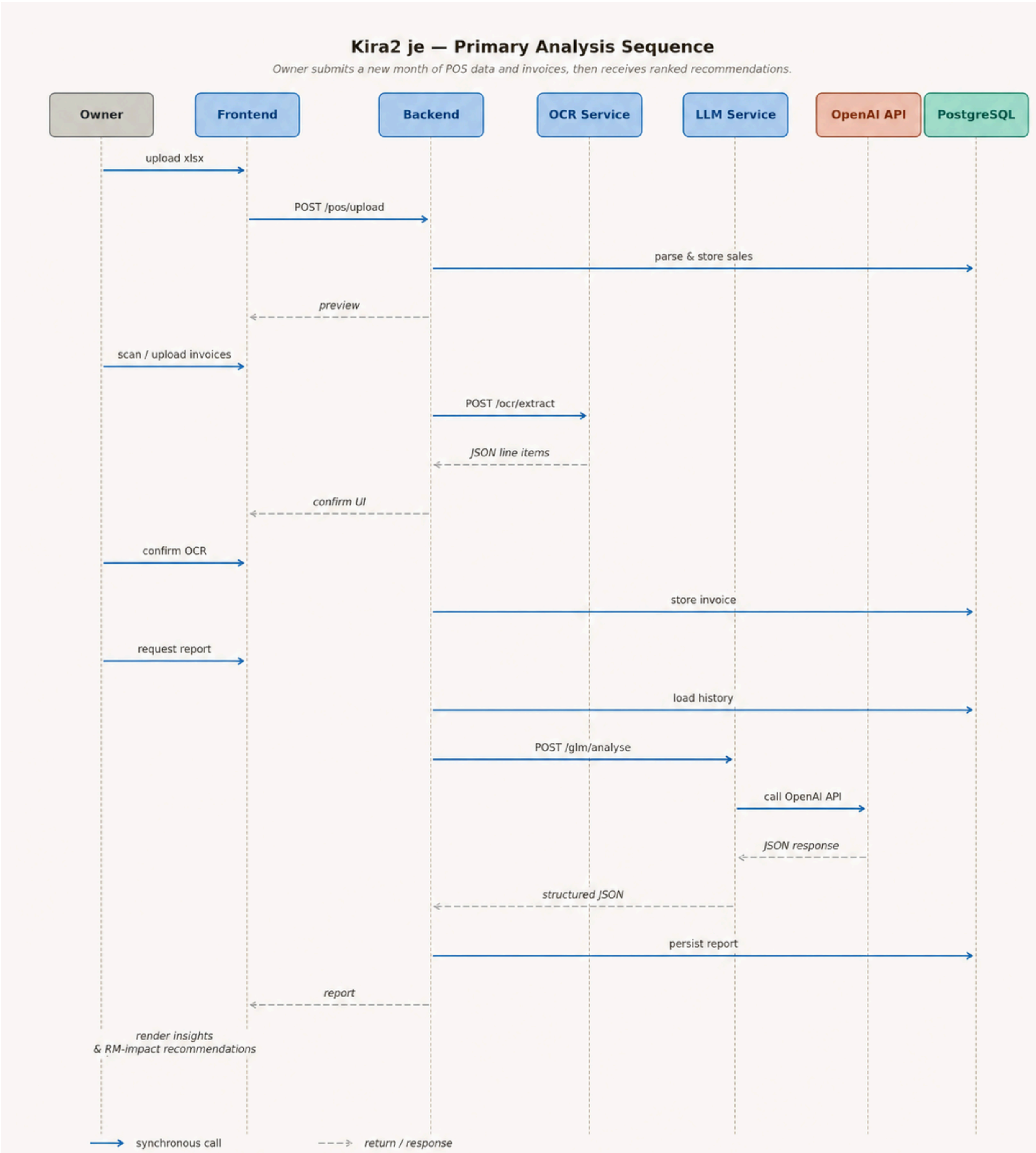

Figure 2.1.8a: Sequence Diagram

2.2 Technological Stack

Layer	Technology & Justification
Frontend	1) Next.js - Allows us to build the frontend and backend into one entity. No need for a separate backend server. 2) React - A library that Next.js is built on. Every other library is chosen based on this framework. Our alternative would be Vue, but some libraries are not as good as React-based libraries. 3) Tailwind CSS - A highly customizable and the go-to option for modern websites as opposed to Bootstrap. 4) Framer Motion - A library that offers a lot of design features for complex animations that would be harder to implement with normal CSS animations. 5) il8n - A module that allows us to use both English and Bahasa Malaysia locales for the website
Backend	1) Node.js - A Javascript runtime environment that enables Javascript to be used in backend server scripts. 2) Typescript - A fork of Javascript that implements syntax. Used as opposed to Python since we do not need any data analytic libraries.
OCR Service	1) Gemini - A free OCR option for our implementation. In this context, we are using Gemini 2.5 Flash, which rate limits to 10 requests per minute and 250 requests per day. 2) Tesseract.js - An OCR library that runs locally that acts as a fallback that takes over if Gemini fails.

	3) Z.AI’s GLM-4.6V - A future enhancement for large scale implementation
LLM Reasoning	1) OpenAI ChatGPT-4o-mini- OpenAI’s flagship model that receives prompts and data from the solution, returns results in JSON format.
File Storage/Database	1) SQLite - Used to run databases locally, does not require any extensive setup and suitable for a Node.js environment. H2 files require Java installed, therefore SQLite is our choice. 2) PostgreSQL - A free option for live deployment. Easy to set up with Vercel. 3) Prisma - An object-relation mapping tool that lets us use Typescript to ‘talk’ to the database instead of using SQL. Prisma supports both SQLite and PostgreSQL. 4) Zod - A Typescript-based library used for declaration and validation. This ensures the data always follows the specified schema.
WhatsApp Channel	1) WhatsApp Business API - Many external vendors offer WhatsApp chat bot capabilities, but this is outside the scope of the preliminary round prototype.
Deployment	1) Vercel - Offers free hosting for our solution, does not need additional complex setup since we are using Node.js 2) neon.tech - A free option for hosting a PostgreSQL database, easy to link up with Vercel
Auth	Phone Number and OTP

2.3 Key Data Flows

2.3.1 Data Flow: Data movement in Kira2 Je is organised around two ingestion channels and one reasoning channel. The two ingestion channels (POS sales and invoice expenses) populate the database independently. The reasoning channel (LLM) reads from the database, performs analysis, and writes report records back to the database. The frontend reads only from persisted reports. It never calls the LLM directly. This separation was done to make the system testable, observable and rollback-safe.

2.3.1.1 Data Flow Diagram (DFD)

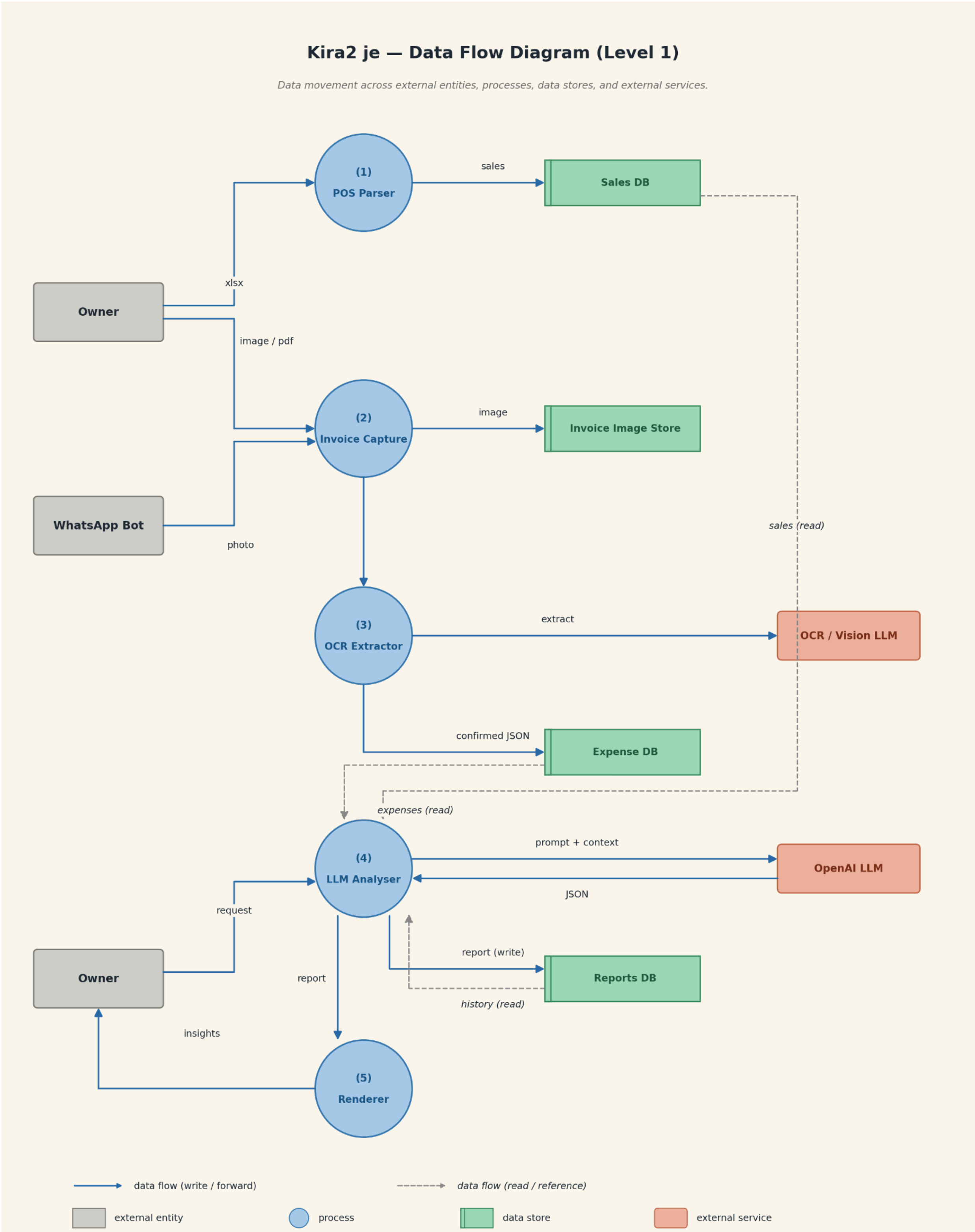

Figure 2.3.1.1a: The OCR Extractor and the LLM Analyser are the only processes that call external services.

Email: umhackathon@um.edu.my

2.3.2 Normalized Database Schema

The logical data model is normalised to the third normal form. The schema is designed so that monthly reports and cannibalisation analysis can be computed without redundant joins, and so that ingredient-to-menu mappings are resolved through an explicit junction table.

Table	**Key Fields & Relationships**
users	user_id (PK), email, password_hash, language_pref (BM/EN), created_at.
businesses	business_id (PK), user_id (FK), business_name, business_type (restaurant/café/catering), pos_system_type.
pos_uploads	upload_id (PK), business_id (FK), file_ref, period_start, period_end, uploaded_at.
sales_items	sales_item_id (PK), upload_id (FK), menu_item_id (FK), quantity, unit_price, sold_at, payment_method, platform.
menu_items	menu_item_id (PK), business_id (FK), item_name, category.
invoices	invoice_id (PK), business_id (FK), supplier_id (FK), invoice_date, invoice_number, image_ref, ocr_confidence, status (draft/confirmed).
suppliers	supplier_id (PK), business_id (FK), supplier_name.
invoice_line_items	line_item_id (PK), invoice_id (FK), description, quantity, unit, unit_price, total.
ingredient_mappings	mapping_id (PK), line_item_id (FK), menu_item_id (FK), allocation_pct. Resolves the M:N relationship between expense line items and menu items.
reports	report_id (PK), business_id (FK), period, generated_at, report_json (full GPT-4o-mini output).
recommendations	rec_id (PK), report_id (FK), rank, title, description, estimated_monthly_impact_rm.

3. Functional Requirements & Scope

This section defines the boundaries of what Kira2 je delivers as a Minimum Viable Product (MVP) for UM Hackathon 2026. The scope is deliberately narrow so that the team can build a high-impact, demonstrable prototype rather than a thinly-spread feature list.

3.1 Minimum Viable Product

3.1.1 Core Features

#	Feature	Description
1	POS Sales Data Upload	The owner exports their POS sales report as .xlsx and uploads it through a drag-and-drop zone or file picker. The backend parses the file, structures it as item-level sales records, and shows a preview before final confirmation.
2	Multi-Channel Invoice Capture	Three invoice input modes feeding a single OCR pipeline: (a) in-app camera for physical invoices, (b) WhatsApp bot for friction-free phone capture, (c) drag-and-drop / file picker for e-invoices and PDFs. Batch upload supported. The owner sees the parsed result and can correct fields before confirmation.
3	GPT-4o-mini Cost-Revenue Analysis	The OpenAI GPT-4o-mini model matches expense line items to menu items, calculates per-item profit after ingredient cost, platform commission and overhead, and returns a structured report including top performers, cost breakdown, items at risk and ranked recommendations with estimated RM impact. Output language follows the user's BM/EN preference.

4	Cannibalisation Detection	After roughly three months of accumulated data, the GPT-4o-mini model identifies cases where a new menu item pulls sales from existing items rather than generating net new revenue. For the hackathon demo this feature is showcased with synthetic multi-month data to prove the concept.
5	What-if Simulator	The owner asks hypothetical questions in natural language ("*What if I raise teh tarik by RM0.50?*" or "*What if I drop this menu item?*"). The GPT-4o-mini model returns projected revenue change, cost change, net RM impact, risks and a recommendation, all in the user's preferred language.

3.2 Non-Functional Requirements (NFRs)

These system qualities are not user-facing features but are crucial to the product's reliability, trustworthiness and economic viability - particularly given that the model is invoked on every analysis call.

Quality	**Requirement**	**Implementation**
Scalability	The system must handle bursts when a batch of invoices is uploaded simultaneously (typical end-of-month behaviour).	Asynchronous job queue for OCR and GPT-4o-mini calls. The frontend shows a progress state while invoices are processed in the background.
Reliability	Owners must trust that an uploaded invoice is not lost. The system targets 99% successful capture for image and PDF submissions.	Atomic write of invoice image to object storage before OCR is invoked. Failed OCR does not delete the source image; the owner can retry extraction without re-uploading.
Maintainability	Each service (Frontend, Backend, OCR, GPT-4o-mini) is independently deployable so that a prompt update does not require a full release.	Service-level versioned prompt templates stored in code and tested in isolation. Independent deployment pipelines for each service.
Token Latency	GPT-4o-mini model analysis calls must return within 8 seconds at p95 under normal load; what-if calls within 5 seconds at p95.	Aggregate sales to item-day granularity before injection. Cap historical context to a rolling summary. Display a user-facing loading state with a progress

		message while awaiting the GPT-4o-mini model's response.
Cost Efficiency	Average GPT-4o-mini token spend per analysis run must stay below a defined RM budget per business per month.	Compact JSON encoding of inputs; chunking by category when payloads are large; 24-hour response cache for repeated queries; aggregated rather than transactional sales.
Localisation	The product must operate fully in BM and English, with the language preference set once at onboarding and applied globally.	User language preference stored on the user record and injected into every GPT-4o-mini prompt. UI strings sourced from a single i18n file per language.
Privacy & Trust	Owners must trust that their sales and supplier data is not exposed to other users.	Per-business row-level isolation in the database. Invoice images are stored under per-business prefixes in object storage. Auth tokens scope all read/write paths.

3.3 Out of Scope / Future Enhancements

The following capabilities are part of the longer-term vision for Kira2 Je but are explicitly not part of the UM Hackathon 2026 MVP.

- Tax calculator and SST/GST reporting: Intentionally dropped from MVP scope to keep the focus on cost-revenue intelligence.
- Inventory management: Kira2 Je is an analysis tool, not an operations tool. Stock levels, reorder points and depletion are out of scope.
- Multi-branch and franchise consolidation: The MVP supports a single business per user account.
- Payment processing: Kira2 Je reads POS data; it does not replace the POS or process transactions.
- Direct POS API integrations: for the MVP, sales data is brought in via .xlsx upload. Native integrations with StoreHub, Slurp, Loyverse and others are future work.
- Advanced authentication (OAuth, SSO, MFA): basic email/password is sufficient for hackathon scope.
- Auto-generated bookkeeping exports for accountants (e.g., AutoCount, SQL Account format) - recognised as valuable but post-MVP.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1 Technical Evaluation

4.1.1. Grayscale Rollout & A/B Testing: When a new prompt template, new GPT-4o-mini version or new analysis feature is introduced, it is released to a small percentage of real businesses initially (target: 5%). Telemetry - recommendation actionability scores, owner thumbs-up/down feedback, latency, error rate - is monitored over a defined window. If no regressions are detected, the rollout is widened to 25%, then 50%, then 100%. A/B testing is also conducted on prompt variants: for example, two phrasings of the recommendation prompt can run in parallel against split traffic, and the variant with higher actionability scores is promoted.

4.1.2 Emergency Rollback (ER) & Golden Release: Every component of Kira2 Je has a designated Golden Release - the last known-good version. If anomalies are detected in production (e.g., GPT-4o-mini JSON parse-failure rate above threshold, OCR confidence collapsing across many invoices, latency spike), the automated pipeline triggers an Emergency Rollback to the Golden Release while engineering investigates. For the GPT-4o-mini service specifically, the Golden Release includes both the previous prompt template and a fallback heuristic summariser that can serve a degraded-but-useful report if the GPT-4o-mini API itself is unreachable.

4.1.3 Priority Matrix

Priority	Trigger Condition	Action
P1 Critical	GPT-4o-mini API failure rate > 2% over a 5-minute window OR JSON parse-failure rate > 5%.	Emergency Rollback to Golden Release. Switch GPT-4o-mini service to fallback summariser. On-call engineer paged.
P1 Critical	OCR confidence median falls below 60% across the rolling 1-hour window.	Halt automatic confirmation; force every owner to manually verify line items. Investigate vision model regression.
P2 High	GPT-4o-mini latency p95 > 12 seconds.	Reduce historical context window; activate response cache more aggressively; investigate OpenAI API status.
P2 High	Recommendation actionability score (owner thumbs-up rate) drops more than 15% week-over-week.	Pause current prompt rollout; revert to previous prompt version; re-evaluate prompt change.
P3 Medium	WhatsApp webhook delivery delay > 30s.	Investigate WhatsApp Business API status; switch to backup webhook channel if configured.
P3 Medium	Language switching produces mixed-language output in the GPT-4o-mini response.	Tighten language directive in the prompt template; redeploy in next minor release.

4.2 Monitoring:

4.2.1 Monitoring Plan

The monitoring plan covers four layers, each tied to alerting thresholds aligned with the priority matrix above.

- **System health:** Backend API availability, response codes, error rates, database connection pool, object storage latency.
- **GPT-4o-mini telemetry:** Per-call latency, token usage, response cost, JSON validation pass-rate, retry count, cache hit-rate.
- **OCR telemetry:** Confidence score histogram, manual-correction rate (a leading indicator of OCR drift), per-supplier accuracy if labels are available.
- **Product feedback:** Owner thumbs-up / thumbs-down on recommendations, and whether a recommendation was acted on (a what-if call referencing a prior recommendation is taken as a proxy for action).

4.3 Assumptions

The following assumptions about operational and environmental conditions hold during development and the hackathon demonstration.

- Owners have stable internet on both desktop and phone during data upload and invoice scanning.
- POS export files (.XLSX) follow reasonably consistent schemas within each major Malaysian POS vendor (StoreHub, Slurp, Loyverse). Per-vendor parsing adapters are written and maintained on the backend.
- Owners have a smartphone with a working rear camera and basic familiarity with WhatsApp.
- OpenAI's GPT-4o-mini API is reachable from our deployment region with acceptable latency, and the model has sufficient comprehension of Malaysian F&B vocabulary in BM and English (e.g., "nasi lemak," "teh tarik," "set").
- Suppliers' invoices are legible - printed invoices, handwritten on standard forms, or PDF e-invoices. Severely damaged or unreadable invoices are an acknowledged edge case handled via manual entry fallback.
- For the hackathon demo, multi-month historical data needed to demonstrate cannibalisation is provided as synthetic sample data, since real users will not have three months of history at the time of judging.

4.4 External Dependencies

The following external tools and services are integrated into the Kira2 Je system. Each carries a risk classification and an associated mitigation strategy.

Tool / Service	**Purpose**	**Risk & Mitigation**
OpenAI's GPT-4o-mini API	Core LLM reasoning engine. Handles prompt inference, multi-step cost-revenue reasoning, structured JSON output, cannibalisation pattern detection and what-if simulation.	Critical. If the GPT-4o-mini API is rate-limited or unavailable, the core product cannot function. Mitigation: response cache (24h), graceful degradation to fallback summariser, exponential-backoff retry, on-call alerting.
Vision-capable OCR Model	Extracts structured line-item data from invoice photos and PDFs across BM, English and mixed-language inputs, including handwritten invoices.	High. OCR drift directly degrades data quality. Mitigation: confidence-score gating, mandatory owner confirmation step, manual-correction feedback loop into evaluation telemetry.
WhatsApp Business API	Receives forwarded invoice photos from owners' phones with zero install friction. Drives habitual usage that powers the cannibalisation feature.	Medium. If unavailable, owners can still use camera and drag-and-drop. Mitigation: clearly communicated alternative input modes; webhook health checks.

Cloud Object Storage	Stores raw invoice images, PDFs and uploaded POS xlsx files. Separates blobs from the relational store.	Medium. Outage prevents new invoice capture but does not affect already-stored data. Mitigation: standard cloud-provider SLA reliance and regional replication for production.
PostgreSQL (managed, e.g., RDS)	Persistent store for users, businesses, sales, invoices, mappings, reports and recommendations.	Critical. Loss of database is loss of multi-month context required for cannibalisation. Mitigation: managed backups, point-in-time recovery.
Authentication Provider	Email/password login with session token for hackathon scope.	Low. Self-hosted in MVP. Future migration to a managed auth provider planned post-hackathon.

Email: umhackathon@um.edu.my

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1 Project Timeline

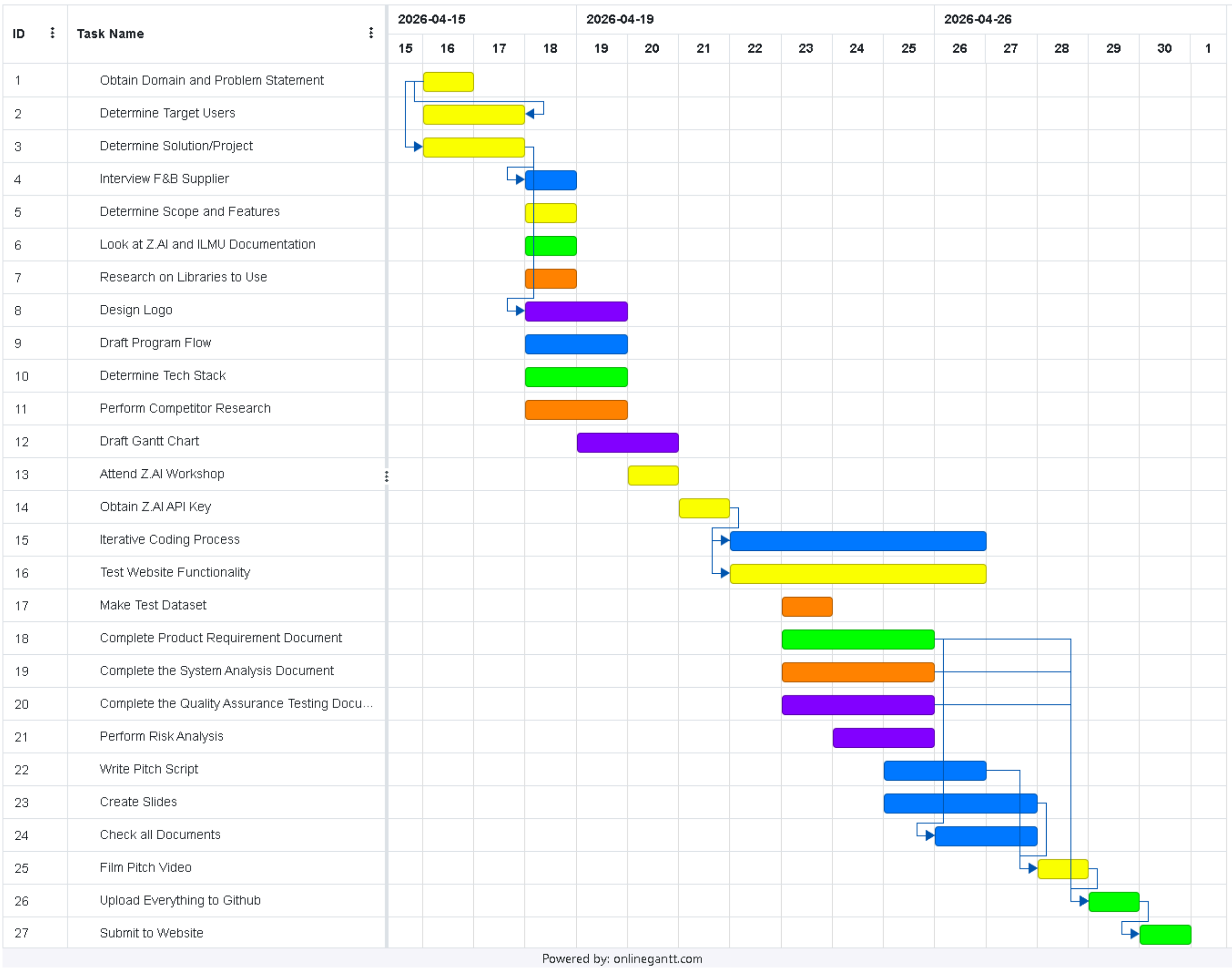

Figure 5.1a: A Gantt chart detailing the timeline of the system development

Member	**Task ID**
Arielle Lee Ern Xin	4, 9, 15, 22, 23, 24
Lee Chong Yew	6, 10, 18, 24, 26, 27
Ummu Kulsum Mahmud	7, 11, 17, 18
Jasmyn Diana binti Darul Farik	8, 12, 20, 21
Everyone	1, 2, 3, 5, 13, 14, 16, 25

5.2 Team Members Role

Member	Role
Arielle Lee Ern Xin	Frontend Lead, Visionary of the Team
Lee Chong Yew	Backend Lead
Ummu Kulsum Mahmud	Data Lead
Jasmyn Diana binti Darul Farik	Creative Design Lead

5.3 Recommendations:

Aspect	Recommendation
Scalability	For large-scale deployment, AWS EC2 can be used, and Amazon has a scheme that allows users to pay according to usage, and its usage can be scaled on demand.
Reliability	When deploying using AWS, ensure that multiple availability zones are selected to provide some redundancy in case one availability zone goes down.
Profitability	Charge users RM5-15 per report generation as a means to make this solution viable by covering the hosting costs.

6. References

Department of Statistics Malaysia. (2024, August 5). *Economic Census 2023: Services statistics – Food and beverage services*. https://www.malaymail.com/news/money/2024/08/05/stats-dept-malaysias-fb-services-gross-output-reaches-rm99b-with-136453-premises-in-2022/146085

Malaysia Digital Economy Corporation. (2026). *National e-invoicing initiative*. https://www.mdec.my/national-einvoicing

Mordor Intelligence. (2026). *Malaysia digital transformation market size & share analysis – Growth trends & forecasts (2026–2031)*. https://www.mordorintelligence.com/industry-reports/malaysia-digital-transformation-market

Slurp Retail Tech Sdn. Bhd. (2026). *Slurp! Point of sale for restaurants & cafés*. https://getslurp.com

SME Corporation Malaysia. (2026). *Challenges in digital adoption*. https://smecorp.gov.my/index.php/en/resources/2015-12-21-10-55-22/news/4461-challenges-in-digital-adoption

StoreHub. (2026). *All-in-one POS system for businesses in Malaysia*. https://www.storehub.com/my/point-of-sale

Email: umhackathon@um.edu.my